

Distributed AI Task Orchestrator (Go)

Portfolio Project Roadmap — Infrastructure-Focused AI Orchestration Platform

■ PRIMARY GOAL: Complete Phases 1–3

Phases 1–3 alone constitute a genuinely impressive portfolio piece. A working distributed task queue in Go with persistence, retry logic, DLQ, and a gRPC/HTTP API demonstrates real distributed systems knowledge. Complete these before considering the rest.

Phase 1 — Core In-Memory Queue [Concurrency Foundations]

- Define Message structure (ID, Type, Payload, Status, Timestamps, RetryCount).
- Implement thread-safe in-memory queue.
- Implement Producers and concurrent Consumers.
- Add graceful shutdown handling.
- Structured logging of message lifecycle.
- Unit tests with race detection (go test -race).

Phase 2 — Persistence & Reliability

- Add durable storage (BoltDB / Badger / PostgreSQL).
- Crash recovery on restart.
- Retry strategies with exponential backoff.
- Dead Letter Queue (DLQ).
- Idempotency keys.
- Visibility timeout concept.

Phase 3 — Networking & Distributed Execution

- Expose HTTP or gRPC API.
- Worker registration and heartbeat.
- Multi-worker processing.
- Basic coordination / leader election mechanism.
- Metrics endpoint (tasks processed, latency, failures).

■ Stop here if needed. Phases 1–3 are a complete, shippable portfolio project.

Continue below only after Phases 1–3 are solid.

Phase 4 — AI Task Execution Engine

- Define AI Task types (Summarize, Classify, Extract, Embed, RAGQuery).
- Provider abstraction layer for LLM APIs (OpenAI, Anthropic, etc.).
- Rate limiting per provider.
- Cost tracking per task.
- Structured JSON output validation.
- Error categorization and retry handling (transient vs. permanent failures).

Phase 5 — RAG Infrastructure Pipeline

- Document ingestion tasks.
- Chunking strategy (fixed-size, sentence-aware).

- Embedding tasks.
- Vector storage abstraction (pgvector, Qdrant, Weaviate).
- Top-K retrieval tasks.
- Context assembly for LLM queries.
- Re-indexing support.

Phase 6 — Evaluation, Monitoring & Observability

- Evaluation task pipeline (prompt regression testing).
- Store metrics (accuracy, latency, cost per task).
- Metrics export (Prometheus-compatible).
- Failure pattern detection.
- Task tracing and execution history.
- Task-level SLA tracking.

Phase 7 — Deployment & DevOps [Leverage Your Azure Background]

Added to the original roadmap. Given your 9 months of Azure/Kubernetes experience, this phase costs you very little effort but significantly rounds out the project for hiring managers.

- Containerize the orchestrator and workers with Docker (multi-stage builds for lean images).
- Write a Helm chart for deploying to Kubernetes.
- Deploy to AKS (Azure Kubernetes Service) — reuse your existing Azure knowledge.
- Configure Horizontal Pod Autoscaler based on queue depth metrics.
- Set up a CI/CD pipeline (GitHub Actions) for build, test, and deploy.
- Add a basic Grafana dashboard using exported Prometheus metrics.

Final Outcome: A production-style Distributed AI Infrastructure Platform demonstrating expertise in Go concurrency, distributed systems design, reliability engineering, modern AI workload orchestration, and cloud-native deployment — with a full DevOps story from code to running cluster.